

Program Analysis 2025-26

Lecture #16

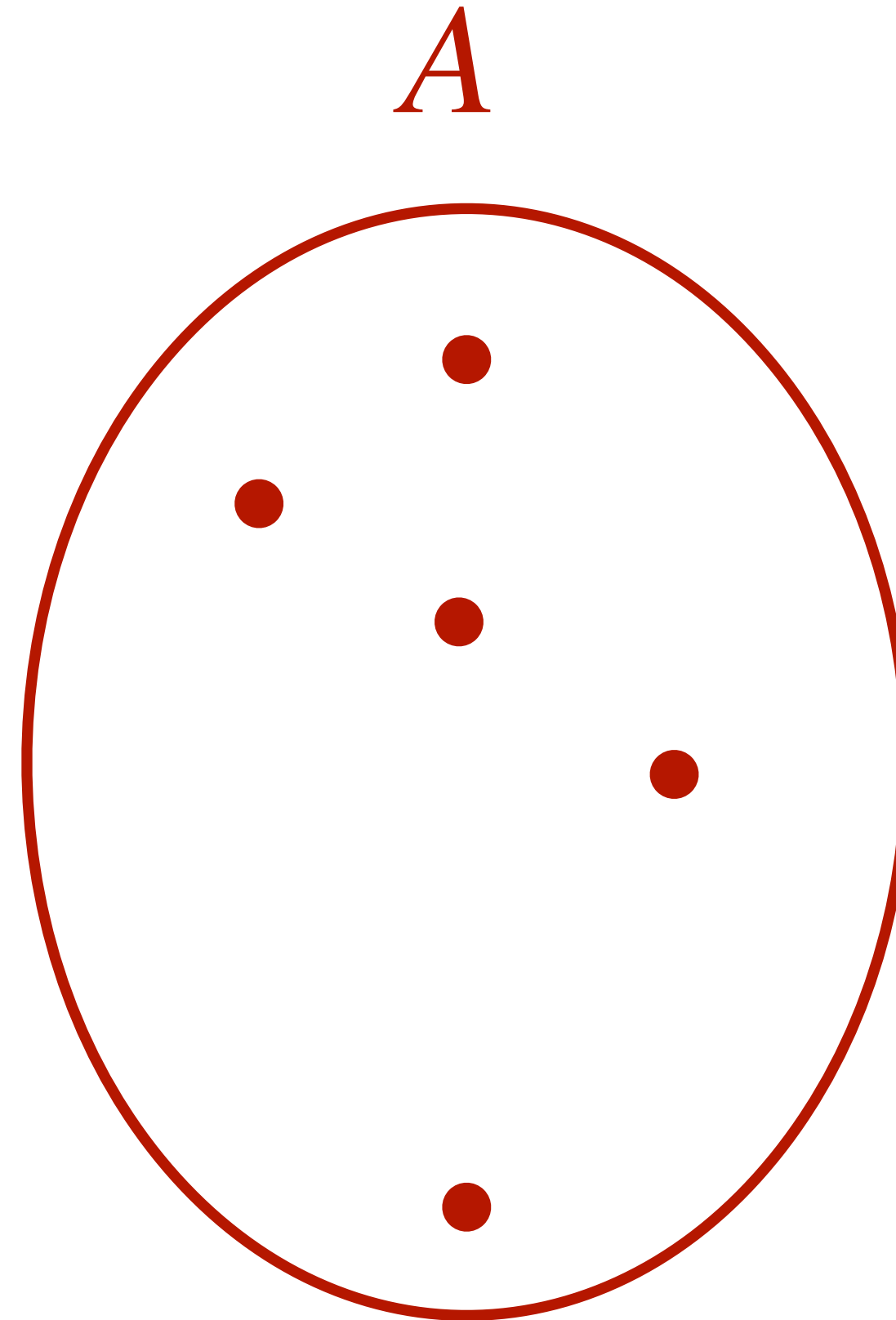
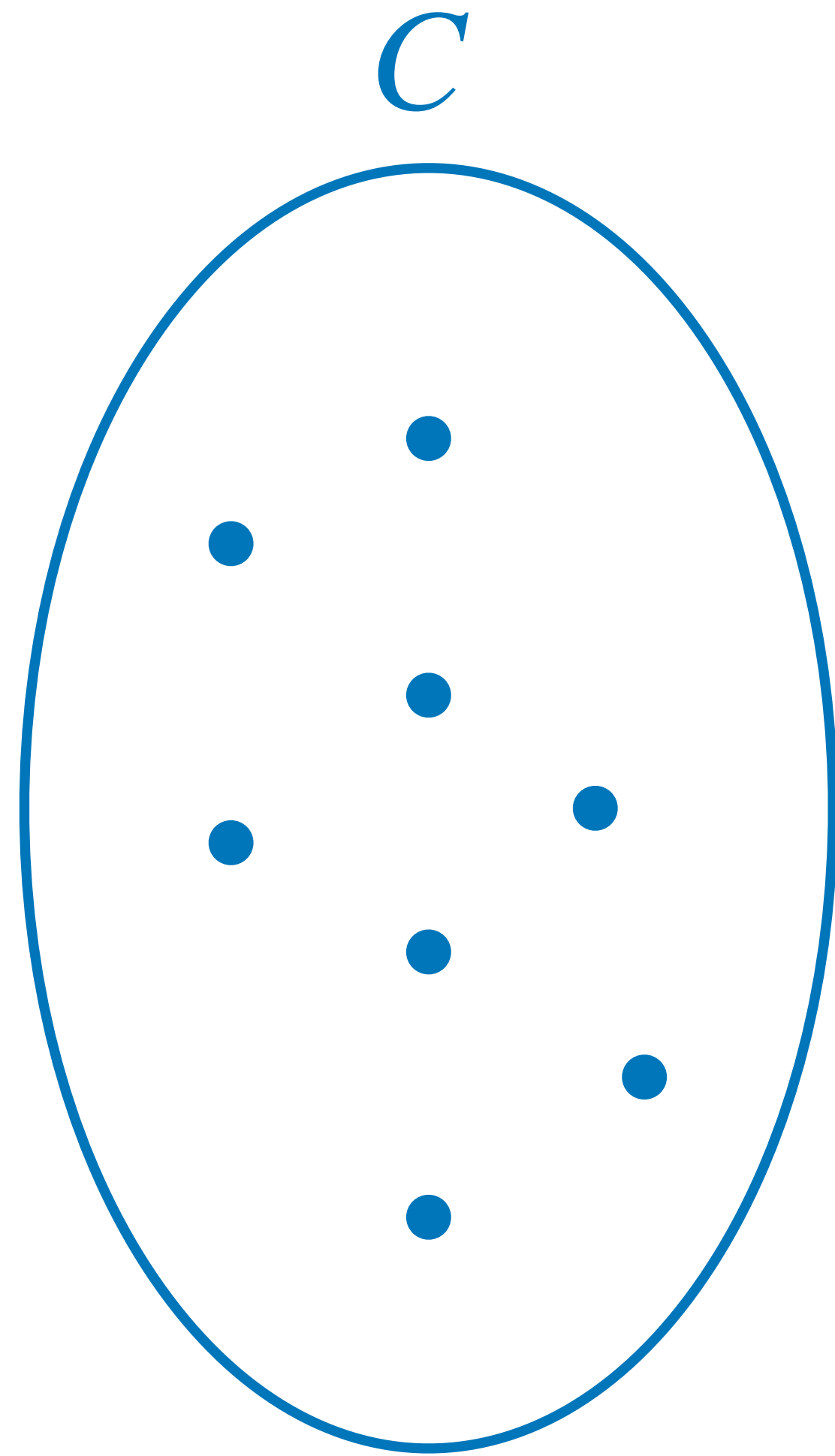
Local completeness logic



UNIVERSITÀ
DI PISA

Recap

Galois insertion as closures



The abstract domain
can just be seen as
a subset of the concrete domain

We write $A(\cdot)$
as a shorthand for $\gamma \circ \alpha$

$A(C)$ is the set of
expressible elements

The map $A : C \rightarrow C$ is a **closure** operator
(monotone, extensive, idempotent)

Completeness, revisited

$$\begin{array}{c} \forall P . A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^{\#} A(P) \\ \subseteq \qquad \qquad \subseteq \\ A(\llbracket c \rrbracket A(P)) \end{array}$$

$\llbracket \cdot \rrbracket$ is the semantics

$\llbracket \cdot \rrbracket^A$ is the bca

$\llbracket \cdot \rrbracket^{\#}$ is the abstract semantics

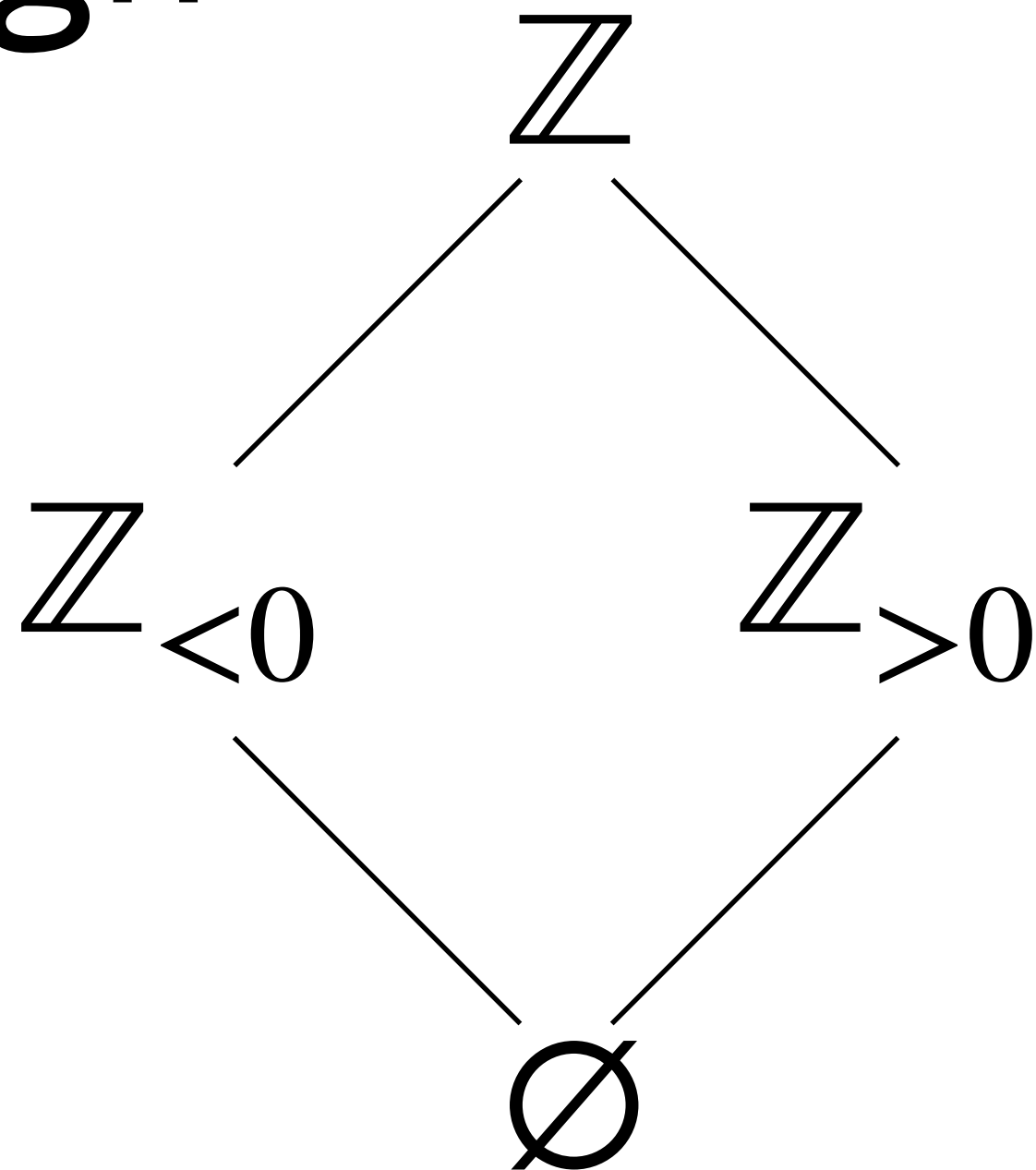
$\llbracket \cdot \rrbracket_A^{\#}$ is also the abstract semantics

Completeness equation: $\forall P . A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket A(P))$

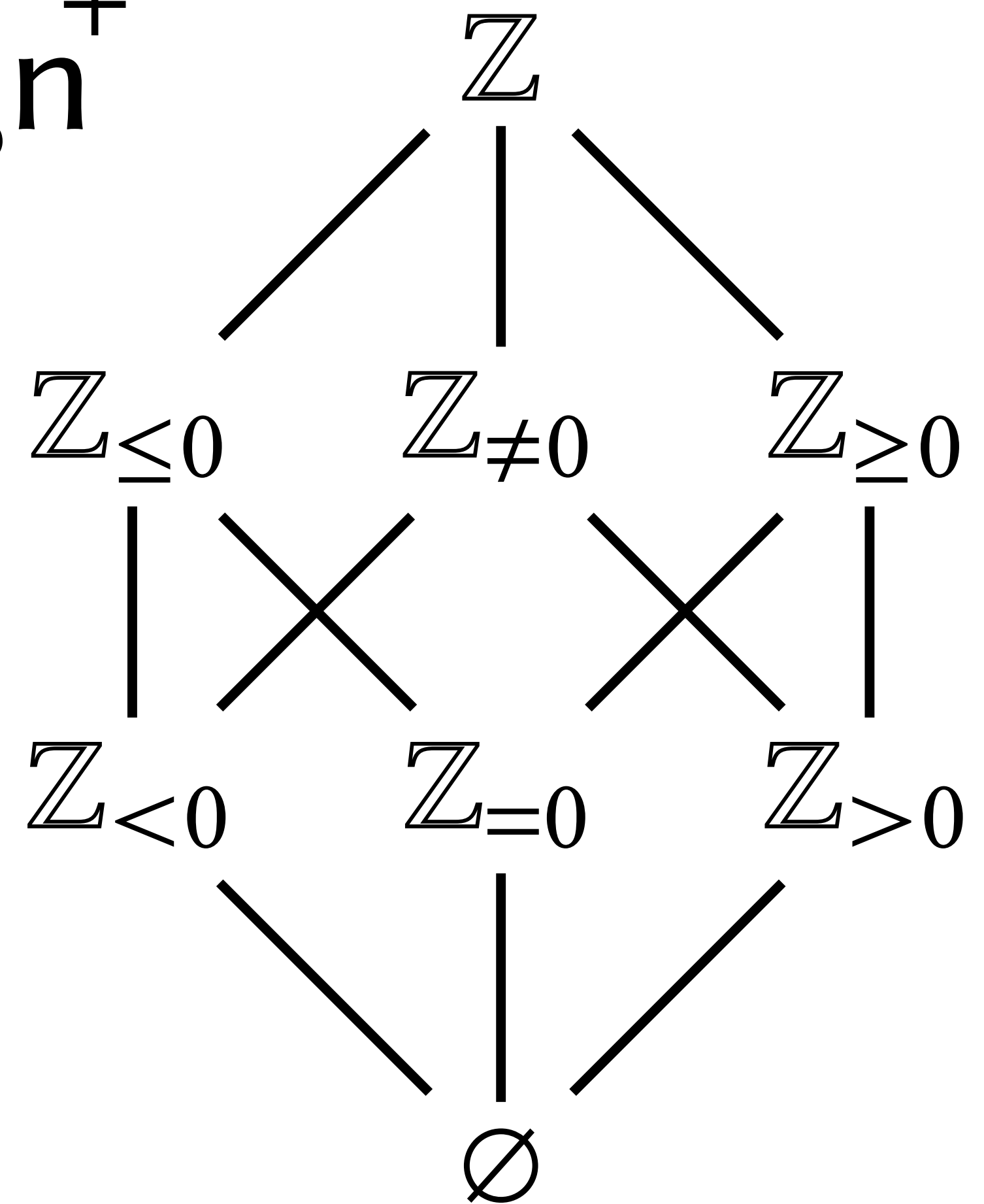
If the bca $\llbracket c \rrbracket^A$ is not complete, then $\llbracket c \rrbracket^{\#}$ is not complete

Sign and Sign⁺

Sign



Sign⁺



Correct or incorrect?

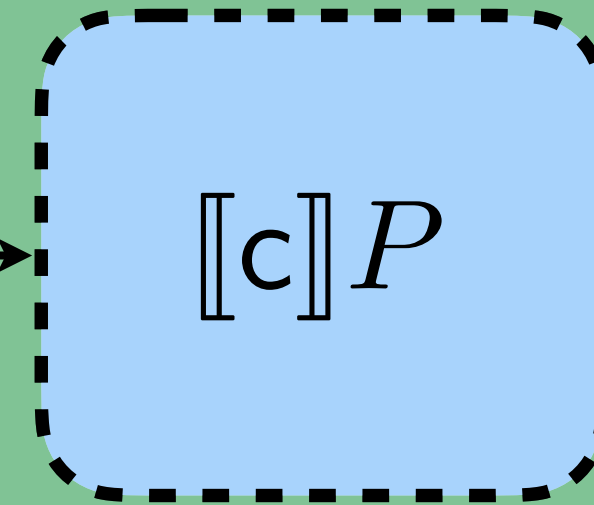
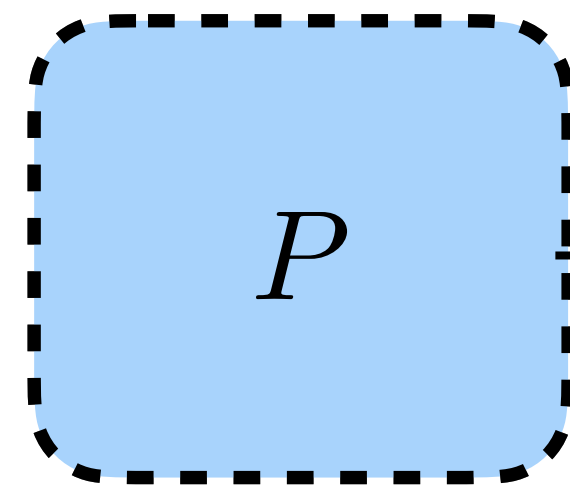
Verification problem

Is it true that, with the given input,
the program will not compute unintended output?



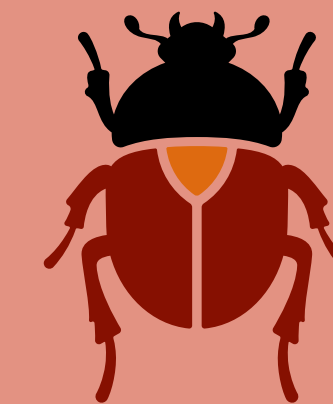
$$\llbracket c \rrbracket P \subseteq Spec$$

Verification problem



$Spec$

$$\llbracket c \rrbracket P \stackrel{?}{\subseteq} Spec$$

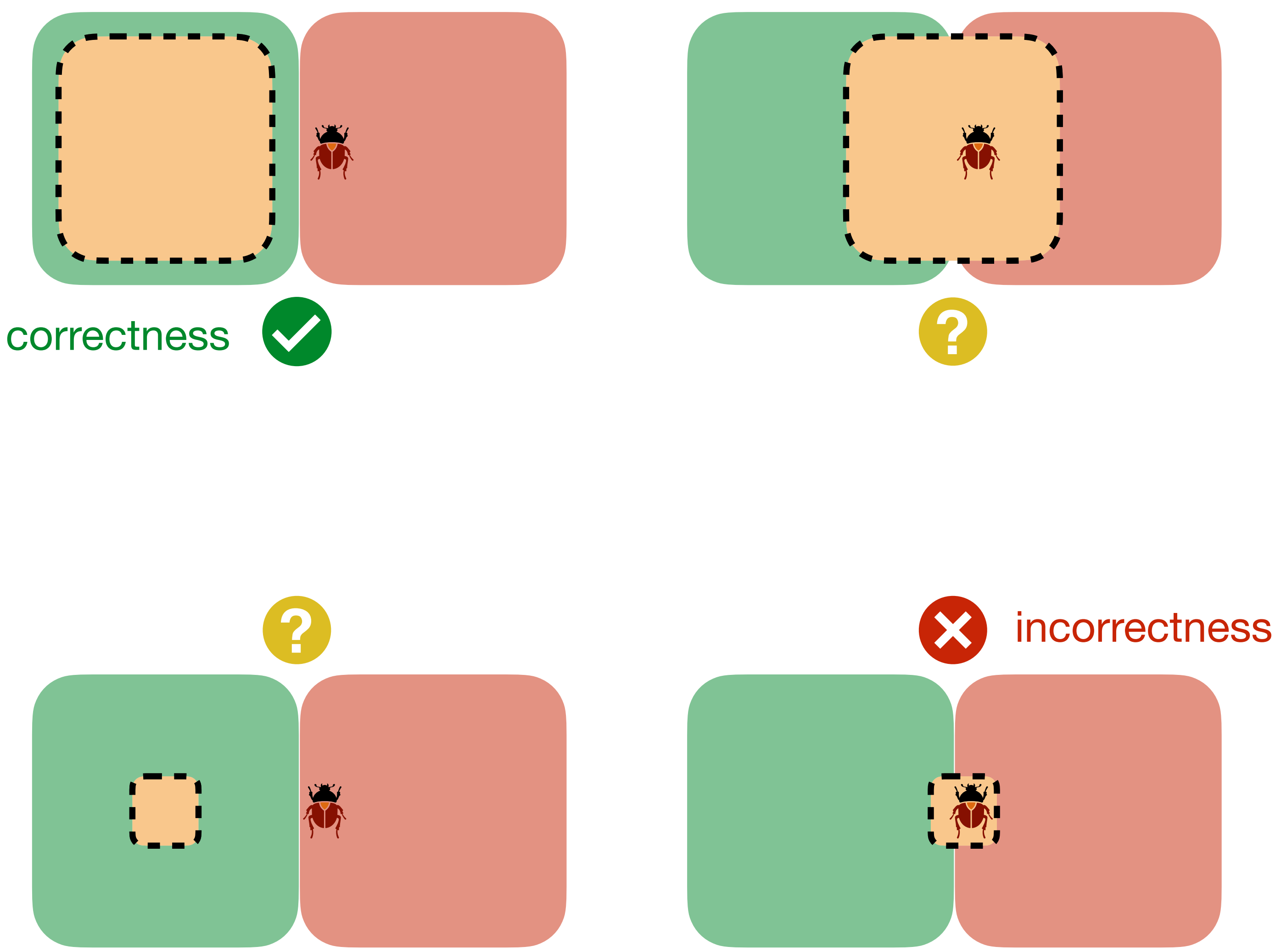
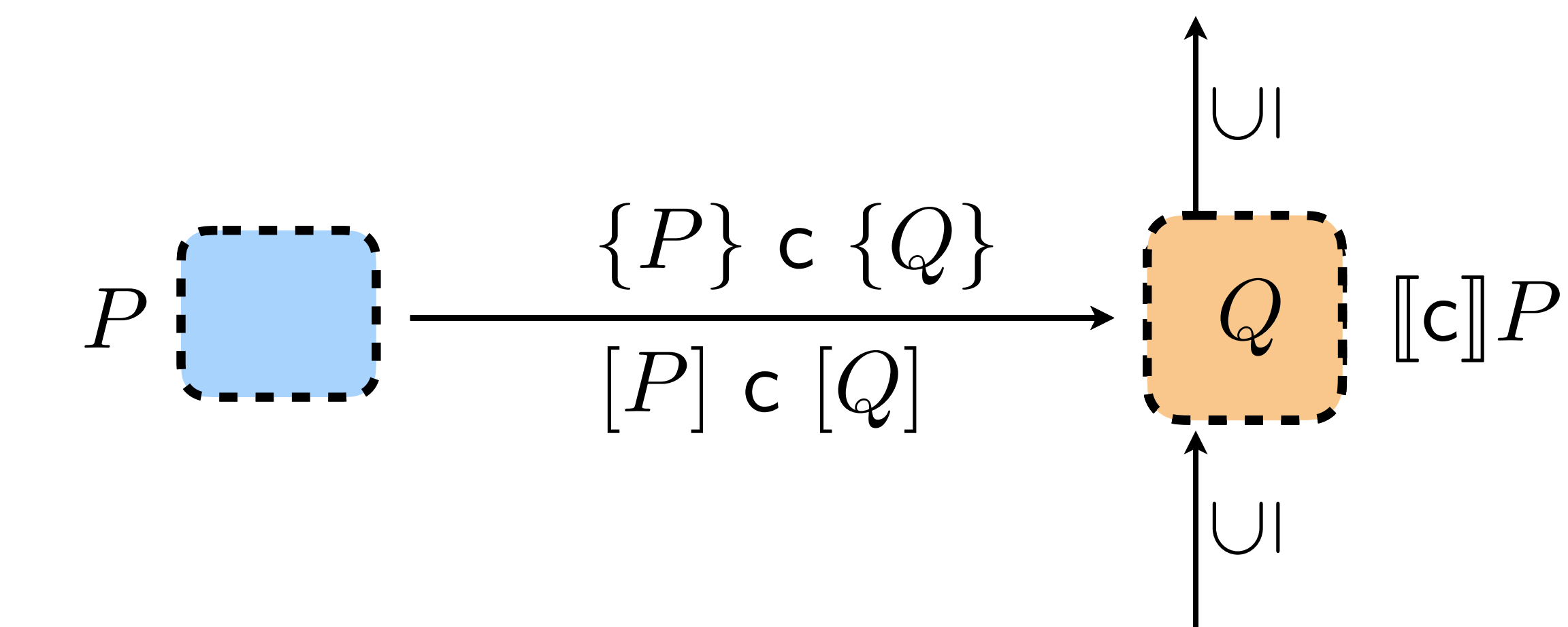


An Axiomatic Basis for Computer Programming

C. A. R. HOARE
The Queen's University of Belfast, Northern Ireland*

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer programs. Examples are given of such axioms and rules, and

Over vs Under

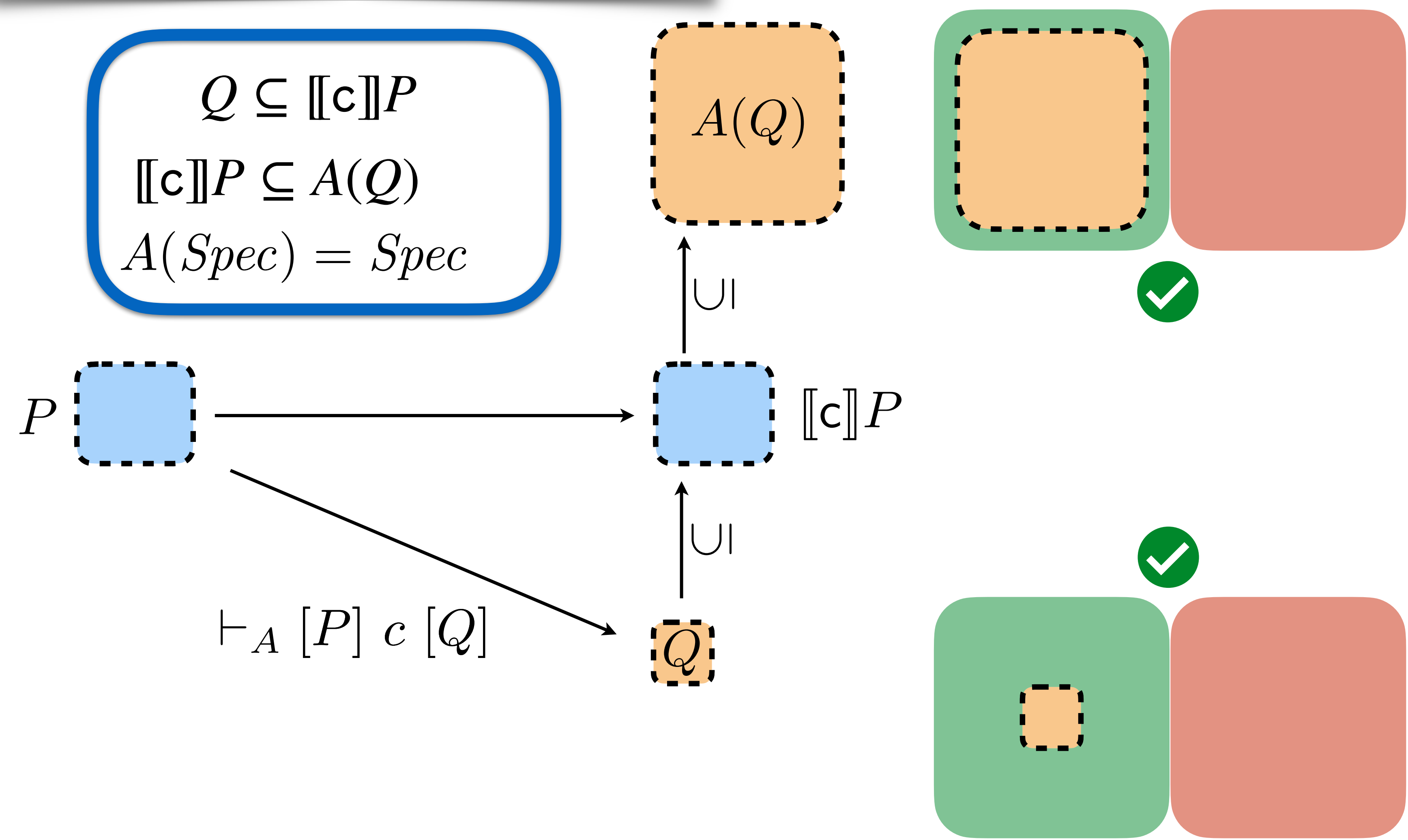


Incorrectness Logic

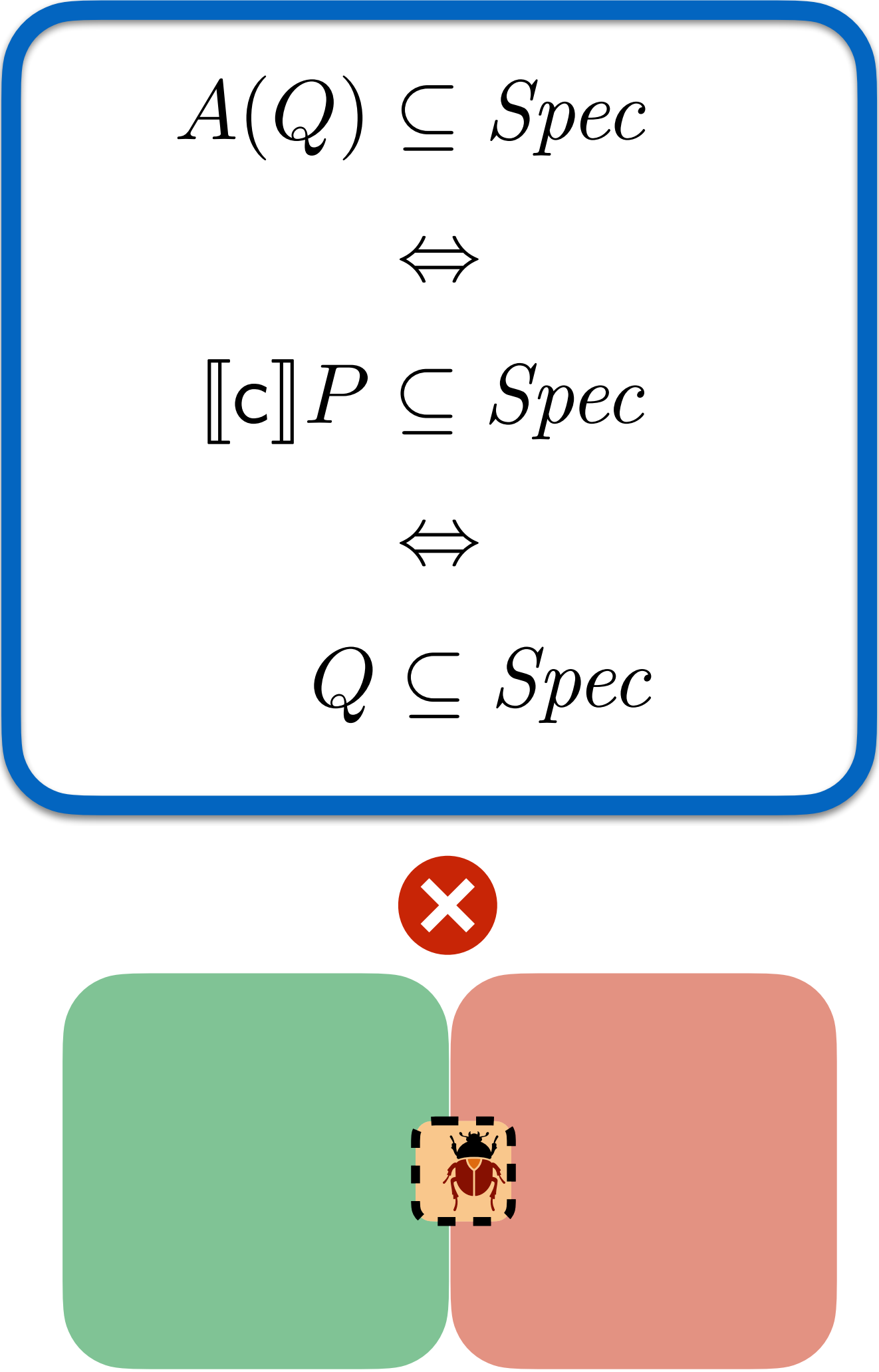
PETER W. O’HEARN, Facebook and University College London, UK

Program correctness and incorrectness are two sides of the same coin. As a programmer, even if you would like to have correctness, you might find yourself spending most of your time reasoning about incorrectness. This includes informal reasoning that people do while looking at or thinking about their code, as well as that supported by automated testing and static analysis tools. This paper describes a simple logic for program incorrectness which is, in a sense, the other side of the coin to Hoare’s logic of correctness.

The idea



Local completeness



A Logic for Locally Complete Abstract Interpretations

Roberto Bruni*, Roberto Giacobazzi[‡], Roberta Gori*, Francesco Ranzato[§]

*University of Pisa, Italy

[‡]University of Verona, Italy

[§]University of Padova, Italy



A Correctness and Incorrectness Program Logic

ROBERTO BRUNI, University of Pisa, Italy

ROBERTO GIACOBAZZI, University of Verona, Italy

ROBERTA GORI, University of Pisa, Italy

FRANCESCO RANZATO, University of Padova, Italy

Abstract interpretation is a well-known and extensively used method to extract over-approximate program invariants by a sound program analysis algorithm. Soundness means that no program errors are lost and it is, in principle, guaranteed by construction. Completeness means that the abstract interpreter reports no false alarms for all possible inputs, but this is extremely rare because it needs a very precise analysis. We introduce a weaker notion of completeness, called *local completeness*, which requires that no false alarms are produced only relatively to some fixed program inputs. Based on this idea, we introduce a program logic, called Local Completeness Logic for an abstract domain A , for proving both the correctness and incorrectness of program specifications. Our proof system, which is parameterized by an abstract domain A , combines over- and under-approximating reasoning. In a provable triple $\vdash_A [p] c [q]$, c is a program, q is an under-approximation of the strongest post-condition of c on input p such that their abstractions in A coincide. This means that q is never too coarse, namely, under some mild assumptions, *the abstract interpretation of c does not yield false alarms for the input p iff q has no alarm*. Therefore, proving $\vdash_A [p] c [q]$ not only ensures that all the alarms raised in q are true ones, but also that if q does not raise alarms, then c is correct. We also prove that if A is the straightforward abstraction making all program properties equivalent, then our program logic coincides with O'Hearn's incorrectness logic, while for any other abstraction, contrary to the case of incorrectness logic, our logic can also establish program correctness.

CCS Concepts: • **Theory of computation** → **Logic and verification**; **Abstraction**; **Programming logic**; **Semantics and reasoning**; **Program analysis**; *Hoare logic*; Axiomatic semantics; **Abstraction**; **Program reasoning**;

Additional Key Words and Phrases: Abstract interpretation, abstract domain, program analysis, program verification, program logic, local completeness, best correct approximation, incorrectness logic

ACM Reference format:

Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2023. A Correctness and Incorrectness Program Logic. *J. ACM* 70, 2, Article 15 (March 2023), 45 pages.

<https://doi.org/10.1145/3582267>

The authors have been funded by the *Italian MIUR*, under the PRIN2017 project no. 201784YSZ5 “Analysis of Program Analyses (ASPRA)” and by a *Meta research* gift. Roberto Giacobazzi and Francesco Ranzato have been partially funded by *Facebook Research*, under a “Probability and Programming Research Award,” by an *Amazon Research Award* for “AWS Automated Reasoning,” and by a *WhatsApp Research Award* on “Privacy-aware Program Analysis.”

Authors' addresses: R. Bruni and R. Gori, University of Pisa, Pisa, Italy; emails: bruni@di.unipi.it, gori@di.unipi.it; R. Giacobazzi, University of Verona, Verona, Italy; email: roberto.giacobazzi@univr.it; F. Ranzato, University of Padova, Padova, Italy; email: ranzato@math.unipd.it.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2023 Copyright held by the owner/author(s).

0004-5411/2023/03-ART15 \$15.00

<https://doi.org/10.1145/3582267>

LICS 2021 + JACM

any locally complete under approximation
either proves the program correct or
incorrect (without false positives)



Local completeness

Expressible specifications

Assume $A(Spec) = Spec$

Take a post $Q \in C$

If $Q \subseteq Spec$

then $A(Q) \subseteq A(Spec) = Spec$

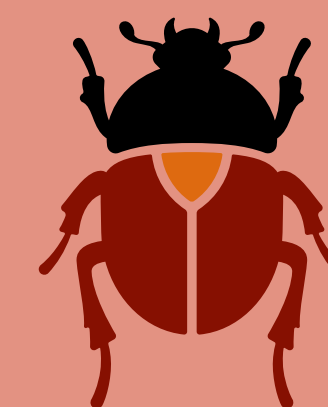
If $Q \not\subseteq Spec$

then $Q \subseteq A(Q) \not\subseteq Spec$

$Q \subseteq Spec \Leftrightarrow A(Q) \subseteq Spec$

Q $Spec$

Q



Example

$$\text{Int}(x \geq 0) = [0, \infty] = (x \geq 0)$$

If $Q_1 \triangleq (x > 0 \wedge x \% 5 = 0)$
then $\text{Int}(Q_1) = [5, \infty] \subseteq (x \geq 0)$

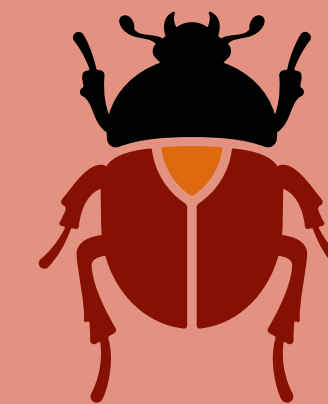
If $Q_2 \triangleq (|x| = 5)$
then $\text{Int}(Q_2) = [-5, 5] \not\subseteq (x \geq 0)$

$$Q \subseteq \text{Spec} \Leftrightarrow A(Q) \subseteq \text{Spec}$$

expressible specification

$$Q_1 \quad x \geq 0$$

Q_2



The role of completeness

correctness of AI
if $\llbracket c \rrbracket_A^\# A(P) \subseteq Spec$
then $\llbracket c \rrbracket P \subseteq Spec$

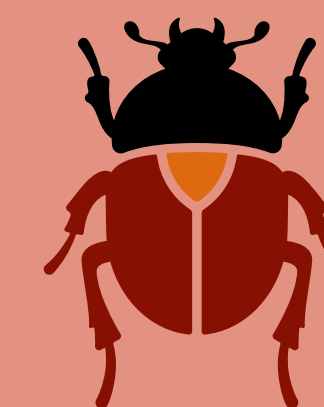
$$\llbracket c \rrbracket P \subseteq A(\llbracket c \rrbracket P) \subseteq \llbracket c \rrbracket_A^\# A(P) \subseteq Spec$$

expressible specification
 $Spec = A(Spec)$

if completeness holds

if $\llbracket c \rrbracket_A^\# A(P) \not\subseteq Spec$
then $\llbracket c \rrbracket P \not\subseteq Spec$

$$\begin{aligned} A(\llbracket c \rrbracket P) = \llbracket c \rrbracket_A^\# A(P) \not\subseteq Spec \\ \Leftrightarrow \llbracket c \rrbracket P \not\subseteq Spec \end{aligned}$$



$$\llbracket c \rrbracket P \subseteq Spec \Leftrightarrow \llbracket c \rrbracket_A^\# A(P) \subseteq Spec$$

Example

$+, \times$ complete in Int

$$c \triangleq x := 3x; x := x + 2$$

$$P \triangleq (x \in \{1, 3, 6\})$$

$$\llbracket c \rrbracket_{\text{Int}}^{\#} \text{Int}(P) = \llbracket c \rrbracket_{\text{Int}}^{\#} [1, 6] = [5, 20] \not\subseteq (x \leq 15)$$

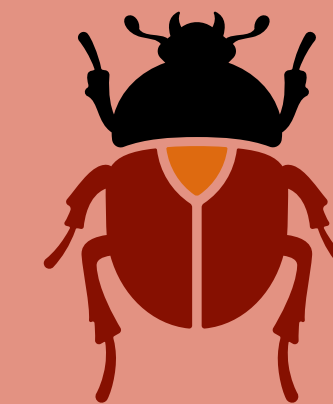


$$\llbracket c \rrbracket P \not\subseteq (x \leq 15)$$

However, not all elements in $[16, 20]$ are true positives!

expressible specification

$$x \leq 15$$



Sources of incompleteness



Completeness is preserved by ; , U and fix

Incompleteness can only be introduced by atomic commands e



assignments: settled for many domains

guards: troublesome

if the bca $\llbracket e \rrbracket^A$ is incomplete, then any (sound) $\llbracket e \rrbracket_A^\#$ is incomplete
i.e., incompleteness is an intrinsic property of a domain

Completeness for guards

Completeness equation: $\forall P. A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$

Lemma. [*a necessary condition for complete guards*]

must be a strict domain

If a test b is complete in A , then b and $\neg b$ are expressible in A

Proof. Assume b not expressible, take $P = b$ and show $\neg b$ is not complete.

Examples

Int: the test $(x = 0)$ is not complete ($x \neq 0$ not expressible)

$[6, +\infty]$ and $[-\infty, 5]$ are both expressible

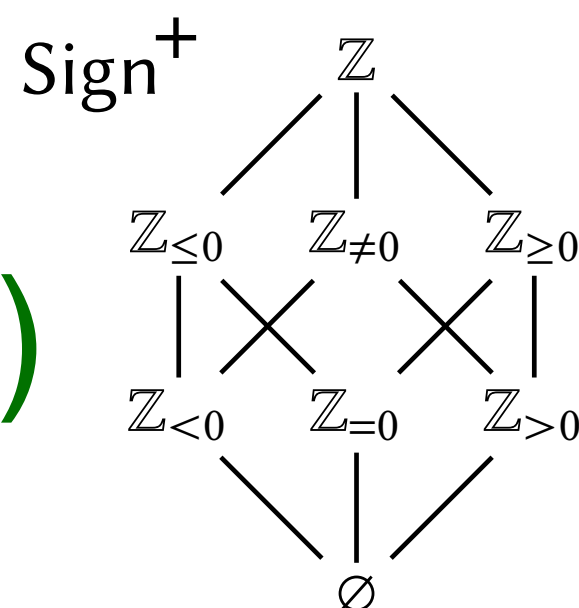
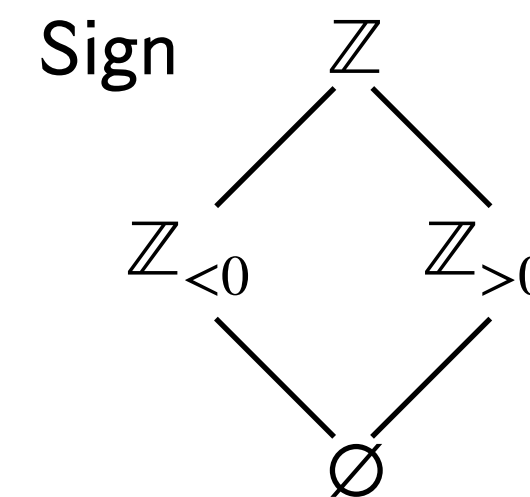
Int: the test $(x > 5)$ might be complete (but it is not)

not expressible

Sign: the test $(x > 5)$ is not complete

$\mathbb{Z}_{>0}$ and $\mathbb{Z}_{\leq 0}$ are both expressible

Sign⁺: the test $(x > 0)$ might be complete (and it is indeed)



Completeness for guards

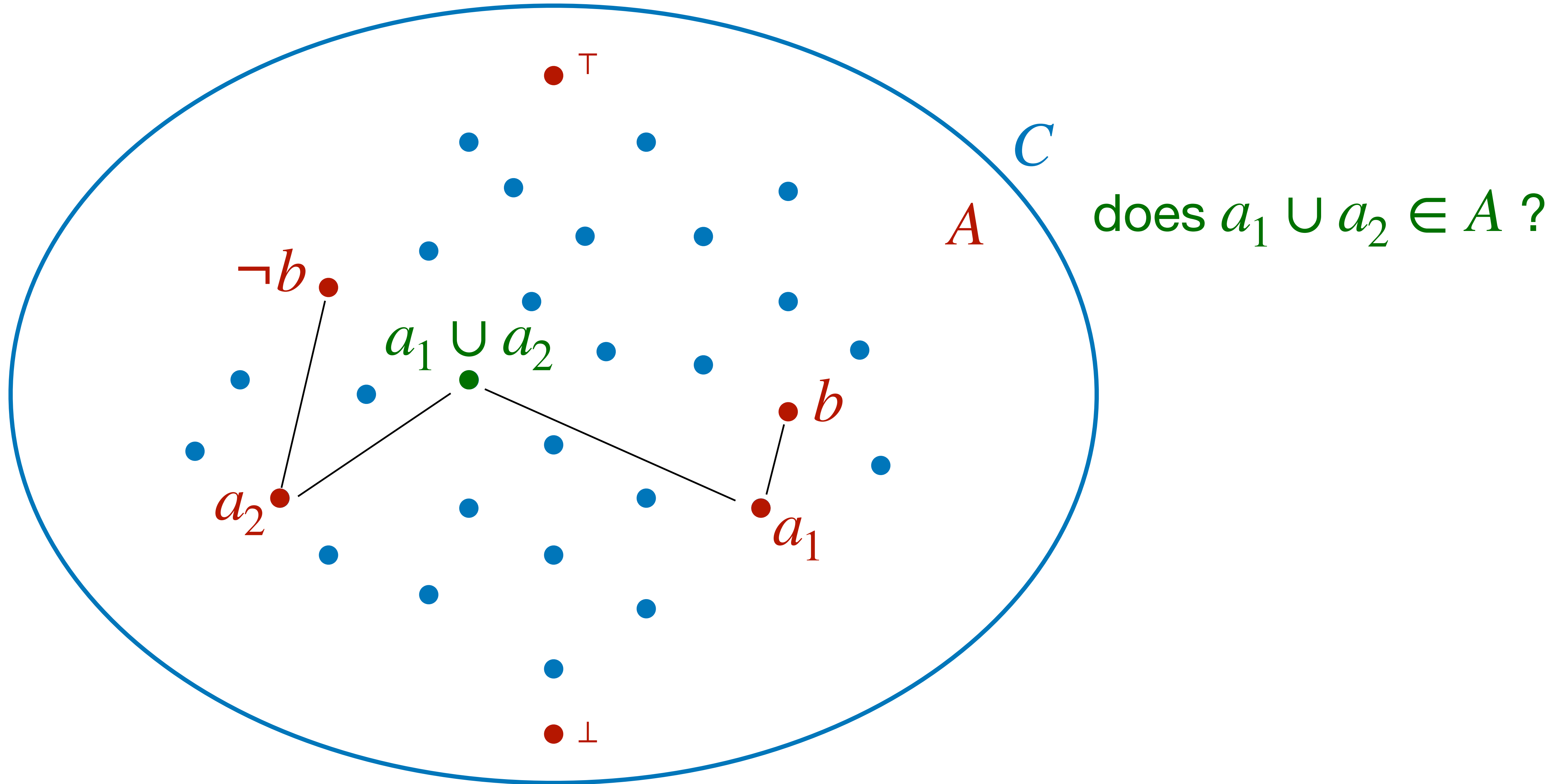
Th. *[a necessary and sufficient condition for complete guards]*

Let b and $\neg b$ be expressible in A .

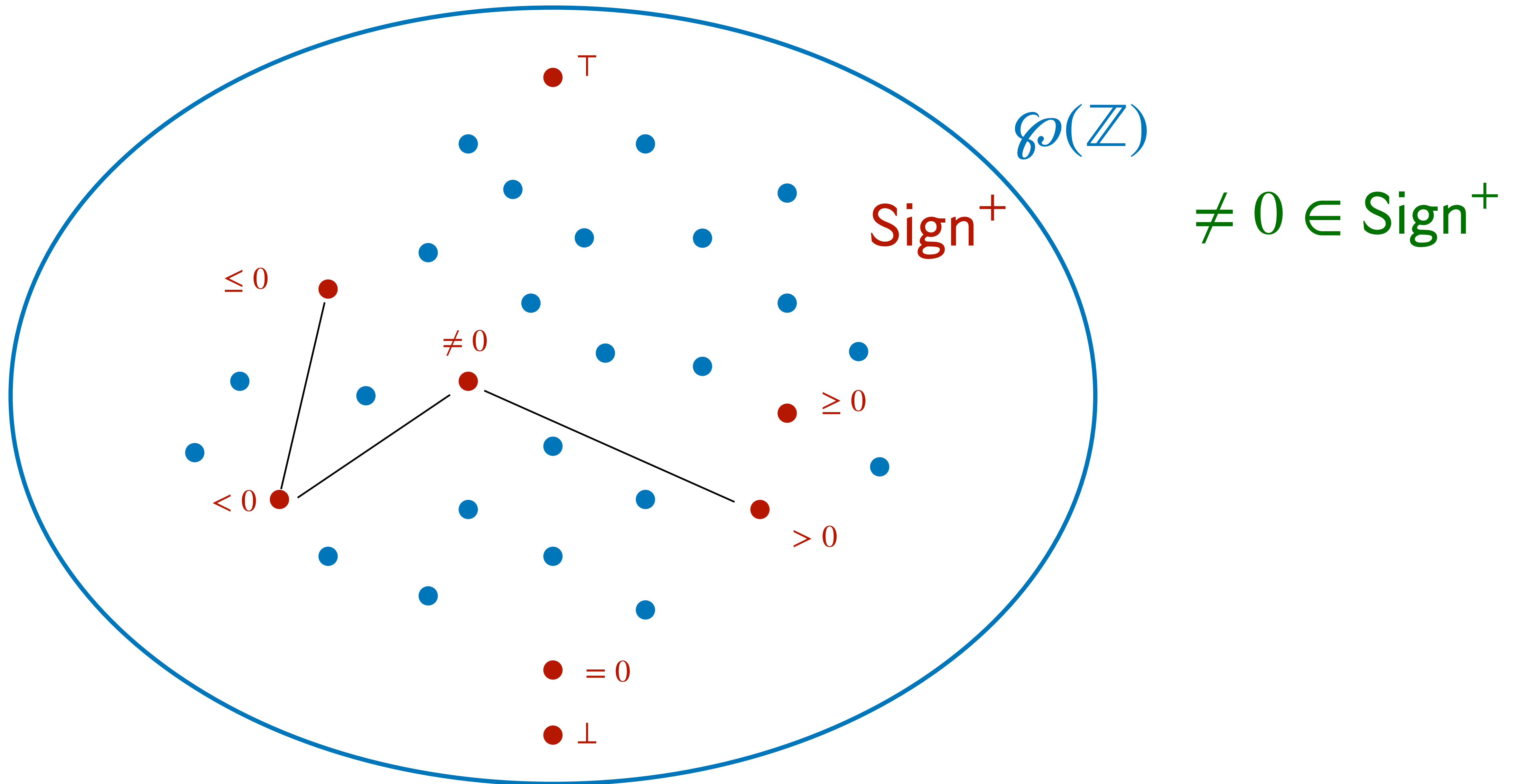
The test b is complete in A
iff

the join of any two abstract points below b and $\neg b$ is expressible.

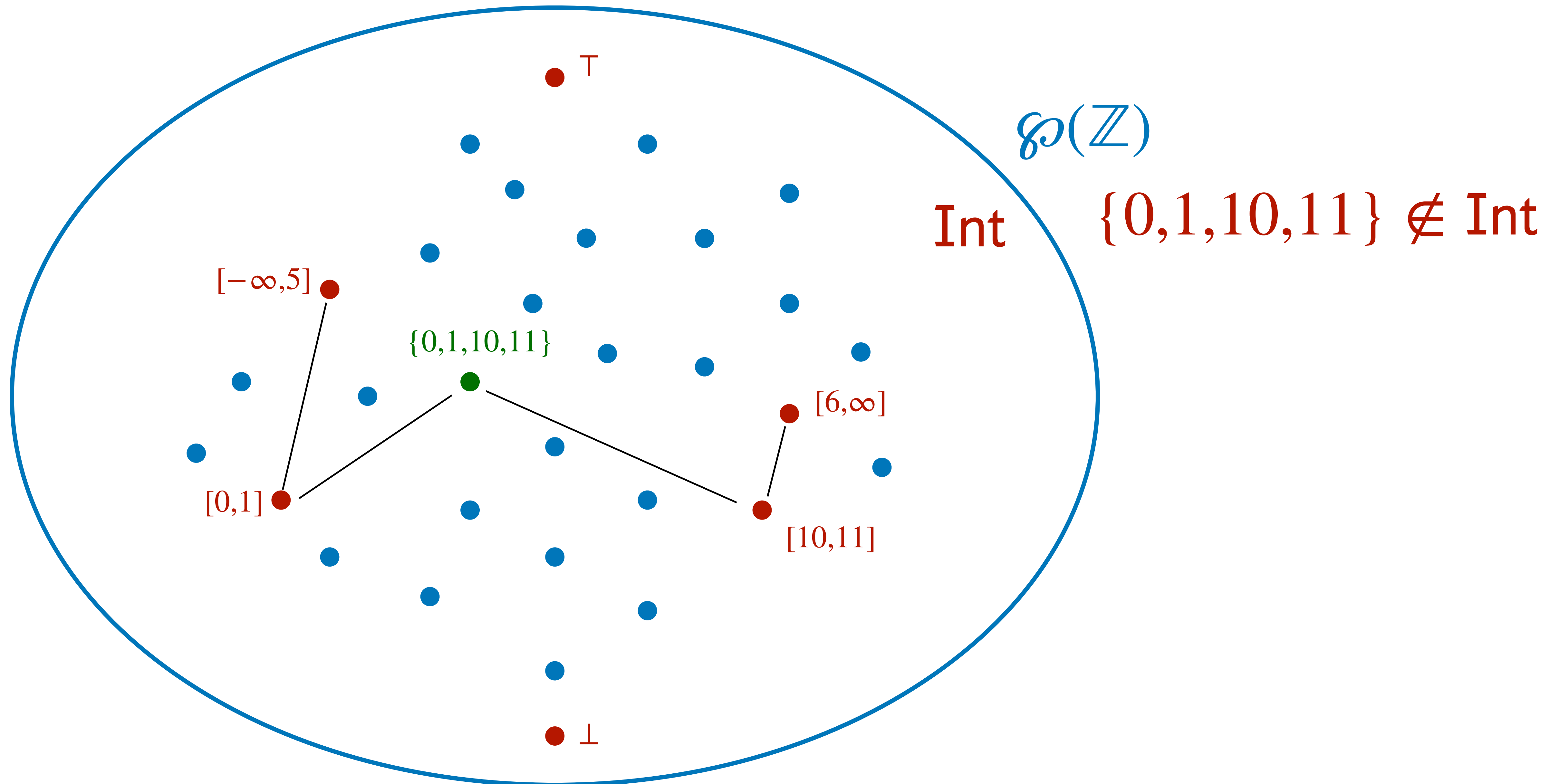
Completeness illustrated



Example



Example

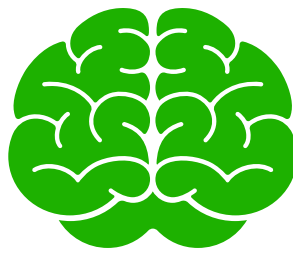


Incompleteness everywhere



Unfortunately, common tests are incomplete in most domains

One possibility:



take the most abstract domain A_b (called complete shell) that:

- 1) refines A , and
- 2) is complete for the test b

ok, but:



may cause a blow up (abstract domains are closed under meet)
operations that were complete in A may be incomplete in A_b

Completeness, revisited

$$\begin{array}{c} \forall P . A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^{\#} A(P) \\ \subseteq \qquad \qquad \subseteq \\ A(\llbracket c \rrbracket A(P)) \end{array}$$

Completeness equation: $\forall P . A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket A(P))$

Local completeness

$$\begin{aligned} \mathbb{C}_P^A(c) &\triangleq A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^\# A(P) \\ &\subseteq \quad \subseteq \\ &A(\llbracket c \rrbracket A(P)) \end{aligned}$$

Local completeness equation: $A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket A(P))$

Local completeness

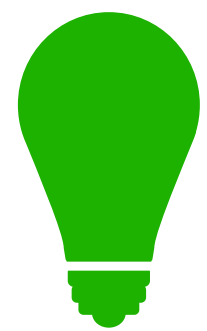
We don't need completeness for all inputs:

e.g., $b \triangleq (x > 0)$ is complete in Int for $P \triangleq \{-10, 0, 1, 10\}$

Local completeness equation: ~~$\forall P. A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$~~

We say that e is locally complete in A for input P and write

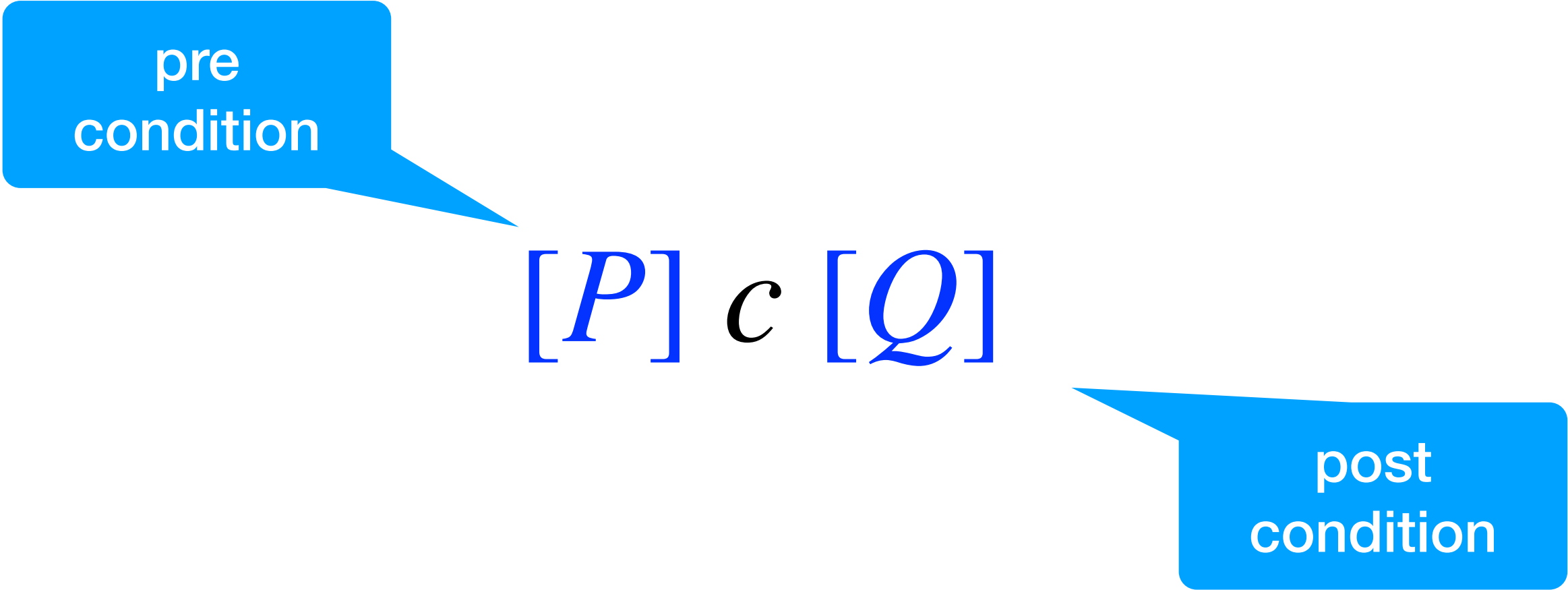
$$\mathbb{C}_P^A(e)$$



Idea: we focus on completeness along traversed states
(which can be collected as underapproximation)

Local Completeness Logic (LCL)

O'Hearn's triples



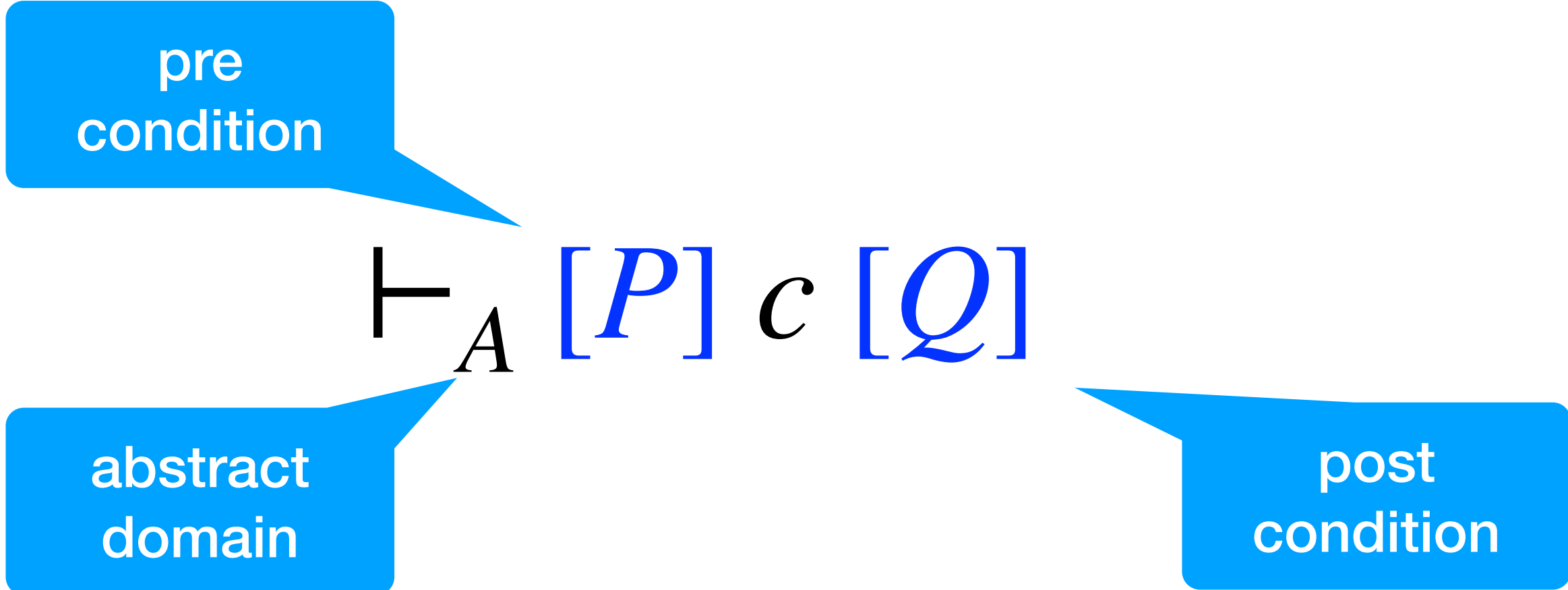
any output matching the postcondition
can be reached by executing the command
on some input matching the precondition

$$[[c]]P \supseteq Q$$

under
approximation!

includes just
reachable states

LCL triples



any output matching the postcondition
can be reached by executing the command
on some input matching the precondition
+
for any input matching the precondition executing the
command establishes the abstraction of the postcondition

can include non
reachable states

$$A(Q) \supseteq [[c]]P \supseteq Q$$

over
approximation!

under
approximation!

includes just
reachable states

Combining under and over approximations



Atomic commands

local
completeness
requirement

$$A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$$

$$\mathbb{C}_P^A(e)$$

$$\frac{}{\vdash_A [P] e [\llbracket e \rrbracket P]} \text{ [transfer]}$$

Floyd's rule
for assignments

$$\mathbb{C}_P^A(a)$$

$$\vdash_A [P] x := a [\exists x'. P[x'/x] \wedge x = a[x'/x]]$$

IL rule
for tests

$$\mathbb{C}_P^A(b)$$

$$\vdash_A [P] b? [P \wedge b]$$

Atomic commands

Floyd's rule
for assignments

$$\mathbb{C}_P^A(a)$$

$$\vdash_A [P] x := a [\exists x'. P[x'/x] \wedge x = a[x'/x]]$$

sum and product
are complete in Int

$$\vdash_{\text{Int}} [x \in \{-7, 7\}] x := 3x + 1 [x \in \{-20, 22\}] ?$$



Atomic commands

IL rule
for tests

$$\frac{\mathbb{C}_P^A(b)}{\vdash_A [P] b? [P \wedge b]}$$

locally
incomplete in Int !

$$\vdash_{\text{Int}} [x \in \{-7, 7\}] x > 0? [x \in \{7\}] ? \quad \text{✗}$$

$$\text{Int}(\llbracket x > 0? \rrbracket \{-7, 7\}) = \text{Int}(\{7\}) = [7, 7] \neq [1, 7] = \text{Int}([1, 7]) = \text{Int}(\llbracket x > 0? \rrbracket [-7, 7]) = \text{Int}(\llbracket x > 0? \rrbracket \text{Int}(\{-7, 7\}))$$

$$\text{Int}(\llbracket x > 0? \rrbracket \{-7, 1, 7\}) = \text{Int}(\{1, 7\}) = [1, 7] = \text{Int}([1, 7]) = \text{Int}(\llbracket x > 0? \rrbracket [-7, 7]) = \text{Int}(\llbracket x > 0? \rrbracket \text{Int}(\{-7, 1, 7\}))$$

$$\vdash_{\text{Int}} [x \in \{-7, 1, 7\}] x > 0? [x \in \{1, 7\}] ? \quad \text{✓}$$

Consequence rule?

$$\frac{P' \Rightarrow P \quad \vdash_A [P'] \ c \ [Q'] \quad Q \Rightarrow Q'}{\vdash_A [P] \ c \ [Q]}$$

IL style
(reversed Hoare)

we can weaken the pre and
shrink the post,
(scalable bug detection)
but not too much!

Convexity

Lemma. [convexity]

If $\mathbb{C}_P^A(\mathbf{e})$ and $P \Rightarrow R \Rightarrow A(P)$ then $\mathbb{C}_R^A(\mathbf{e})$

Proof.

Assume $A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$

we want to prove $A(\llbracket e \rrbracket R) = A(\llbracket e \rrbracket A(R))$

$$A(\llbracket e \rrbracket P) \leq A(\llbracket e \rrbracket R) \leq A(\llbracket e \rrbracket A(R)) = A(\llbracket e \rrbracket A(P)) = A(\llbracket e \rrbracket P)$$

Consequence rule...

 $P' \Rightarrow P$ $\vdash_A [P'] \ c \ [Q'] \quad Q \Rightarrow Q'$ $\vdash_A [P] \ c \ [Q]$

[relax]

IL style
(reversed Hoare)

Consequence rule

preserve
abstraction

preserve
abstraction

$$P' \Rightarrow P \Rightarrow A(P') \quad \vdash_A [P'] \ c \ [Q'] \quad Q \Rightarrow Q' \Rightarrow A(Q)$$

$$\vdash_A [P] \ c \ [Q]$$

[relax]

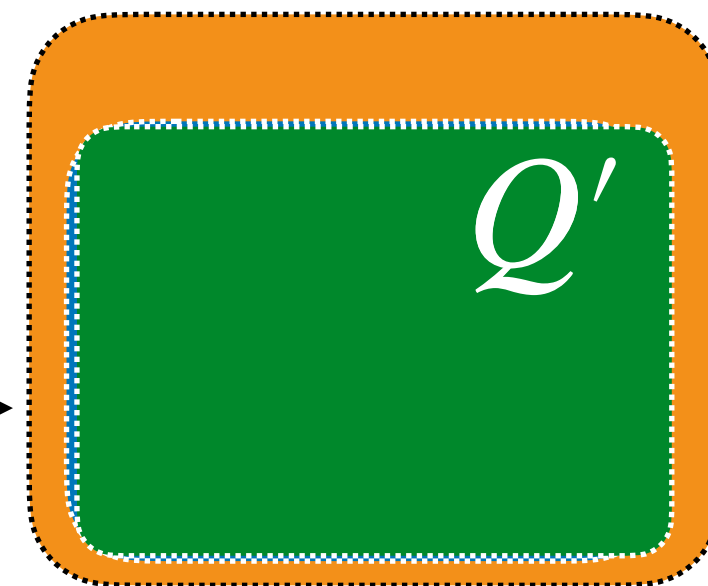
IL style
(reversed Hoare)

$$A(P') = A(P)$$

$$A(Q) = A(Q')$$



$$\vdash_A [P'] \ c \ [Q]$$



Consequence rule

preserve
abstraction

preserve
abstraction

$$\frac{P' \Rightarrow P \Rightarrow A(P') \quad \vdash_A [P'] \ c \ [Q'] \quad Q \Rightarrow Q' \Rightarrow A(Q)}{\vdash_A [P] \ c \ [Q]} \text{ [relax]}$$

IL style
(reversed Hoare)

$$\frac{\vdash_{\text{Int}} [x \in \{-7, 0, 7\}] \ c \ [x \in \{-5, -2, 8\}]}{\vdash_{\text{Int}} [x \in \{-7, 0, 3, 7\}] \ c \ [x \in \{-5, 8\}]}$$



Consequence rule

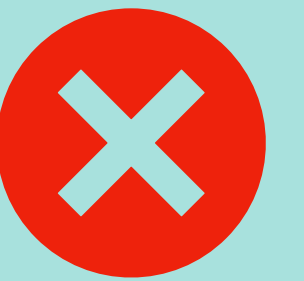
preserve
abstraction

preserve
abstraction

$$\frac{P' \Rightarrow P \Rightarrow A(P') \quad \vdash_A [P'] \ c \ [Q'] \quad Q \Rightarrow Q' \Rightarrow A(Q)}{\vdash_A [P] \ c \ [Q]} \text{[relax]}$$

IL style
(reversed Hoare)

$$\frac{\vdash_{\text{Int}} [x \in \{-7, 0, 7\}] \ c \ [x \in \{-5, -2, 8\}]}{\vdash_{\text{Int}} [x \in \{-7, 0, 7, 9\}] \ c \ [x \in \{-2, 8\}]}$$



Fixpoint acceleration

one-step
unroll

abstract
fixpoint!

$$\begin{array}{c}
 \frac{\vdash_A [P] \text{ } c \text{ } [R] \quad \vdash_A [P \vee R] \text{ } c^\star [Q]}{[\text{rec}] \quad \vdash_A [P] \text{ } c^\star [Q]} \quad \frac{\vdash_A [P] \text{ } c \text{ } [Q] \quad Q \Rightarrow A(P)}{[\text{iterate}] \vdash_A [P] \text{ } c^\star [P \vee Q]}
 \end{array}$$

locally complete under-approximation!
scalable bug detection

$$\begin{array}{c}
 \frac{\frac{\mathbb{C}_P^{\text{Sign}^+}(\llbracket x \leq 0? \rrbracket)}{\vdash_{\text{Sign}^+}[P] \text{ } x \leq 0? [\{-10, -1\}]} \quad (\text{transfer}) \quad \frac{\mathbb{C}_{\{-10, -1\}}^{\text{Sign}^+}(\llbracket x := x * 10 \rrbracket)}{\vdash_{\text{Sign}^+}[\{-10, -1\}] \text{ } x := x * 10 [\{-100, -10\}]} \quad (\text{transfer})}{\vdash_{\text{Sign}^+}[P] \text{ } x \leq 0?; x := x * 10 [\{-100, -10\}] \quad \{-100, -10\} \subseteq \text{Sign}^+(P) = \mathbb{Z}_{\neq 0}} \quad (\text{seq}) \\
 \hline
 \vdash_{\text{Sign}^+}[P] \text{ } (x \leq 0?; x := x * 10)^* [\{-100, -10, -1, 100\}] \\
 P \triangleq \{-10, -1, 100\}
 \end{array}$$

(iterate)

Sequential composition

$$\frac{\vdash_A [P] c_1 [R] \quad \vdash_A [R] c_2 [Q]}{\vdash_A [P] c_1; c_2 [Q]} [\text{seq}]$$

$$\begin{array}{c} [x \in [-\infty, -1]] \\ | \\ \vdash_{\text{Int}} [\text{true}] x < 0? ; x := -x [x \in [1, \infty]] \end{array}$$

Choice

$$\frac{\vdash_A [P] c_1 [Q_1] \quad \vdash_A [P] c_2 [Q_2]}{\vdash_A [P] c_1 + c_2 [Q_1 \vee Q_2]} \text{[join]}$$

$$\vdash_{\text{Sign}} [\text{true}] \text{ if } x < 0 \text{ then } x := -x \text{ else skip } [x \in \{0, 42\}]$$

Diagram illustrating the choice operation with annotations:

- Annotation $\mathbb{Z}_{<0}$ is above the condition $x < 0$.
- Annotation $[x = -42]$ is above the assignment $x := -x$.
- Annotation $\mathbb{Z}_{>0}$ is above the condition $x > 0$.
- Annotation $[x = 42]$ is above the assignment $x := -x$.
- Annotation $\mathbb{Z}_{\geq 0}$ is above the skip statement.
- Annotation $[x \in \{0, 100\}]$ is below the assignment $x := -x$.
- Annotation $\mathbb{Z}_{\geq 0}$ is below the skip statement.
- Annotation $[x \in \{0, 100\}]$ is below the skip statement.
- Annotation $\mathbb{Z}_{\geq 0}$ is below the skip statement.

**Validity, soundness,
completeness**

The rules of LCL

$$\frac{\mathbb{C}_P^A(e)}{\vdash_A [P] e [[e]]P} \text{ (transfer)} \quad \frac{P' \leq P \leq A(P') \quad \vdash_A [P'] r [Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P] r [Q]} \text{ (relax)}$$

$$\frac{\vdash_A [P] r_1 [R] \quad \vdash_A [R] r_2 [Q]}{\vdash_A [P] r_1; r_2 [Q]} \text{ (seq)} \quad \frac{\vdash_A [P] r_1 [Q_1] \quad \vdash_A [P] r_2 [Q_2]}{\vdash_A [P] r_1 \oplus r_2 [Q_1 \vee Q_2]} \text{ (join)}$$

$$\frac{\vdash_A [P] r [R] \quad \vdash_A [P \vee R] r^* [Q]}{\vdash_A [P] r^* [Q]} \text{ (rec)} \quad \frac{\vdash_A [P] r [Q] \quad Q \leq A(P)}{\vdash_A [P] r^* [P \vee Q]} \text{ (iterate)}$$

Auxiliary rules

$$\frac{\vdash_A [P] r [Q] \quad Q \leq P}{\vdash_A [P] r^* [P]} \text{ (invariant)}$$

$$\frac{\vdash_A [P] r [Q] \quad A(P) = A(Q)}{\vdash_A [P] r^* [Q]} \text{ (abs-fix)}$$

$$\frac{\forall n \in \mathbb{N}. \vdash_A [P_n] r [P_{n+1}]}{\vdash_A [P_0] r^* [\bigvee_{i \in \mathbb{N}} P_i]} \text{ (limit)}$$

Validity

A LCL triple $\vdash_A [P] c [Q]$ is **valid** if $Q \subseteq \llbracket c \rrbracket P \subseteq A(Q)$

Is $\vdash_{\text{Int}} [x > 0] x := 10x [x \geq 10]$ valid? 

Is $\vdash_{\text{Int}} [x > 0, y \geq 0] x := yx [x \geq 0]$ valid? 

Is $\vdash_{\text{Sign}} [x > 0, y > 0] x := yx [x = 42, y = 7]$ valid? 

Is $\vdash_{\text{Sign}} [x > 0] (x := x + 1)^* [x > 0]$ valid? 

Logical correctness

Th.

If $\vdash_A [P] \ c \ [Q]$ then $Q \subseteq \llbracket c \rrbracket P \subseteq A(Q) = \llbracket c \rrbracket_A^\# A(P)$

Proof.

By induction on the derivation.

Verification

Th.

If $A(\textit{Spec}) = \textit{Spec}$, then any provable triple $\vdash_A [P] \ c \ [Q]$ either shows the program correct ($Q \subseteq \textit{Spec}$) or exposes some true positives ($Q \setminus \textit{Spec} \neq \emptyset$)

bug finding!

Proof.

$$\begin{aligned} [[c]]P \subseteq \textit{Spec} &\Leftrightarrow A[[c]]P \subseteq \textit{Spec} \\ &\Leftrightarrow A(Q) \subseteq \textit{Spec} \\ &\Leftrightarrow Q \subseteq \textit{Spec} \end{aligned}$$

Logical completeness

Th.

If A is complete for any atomic command in c ,
then any valid triple $\vdash_A [P] c [Q]$ can be derived

Proof.

We first derive $\vdash_A [P] c [[c]]P$,
then use [relax] with $Q \subseteq [[c]]P$

Intrinsic logical incompleteness

Th.

For any Turing complete language and any non-trivial abstraction A , there are valid triples that cannot be proved

Proof.

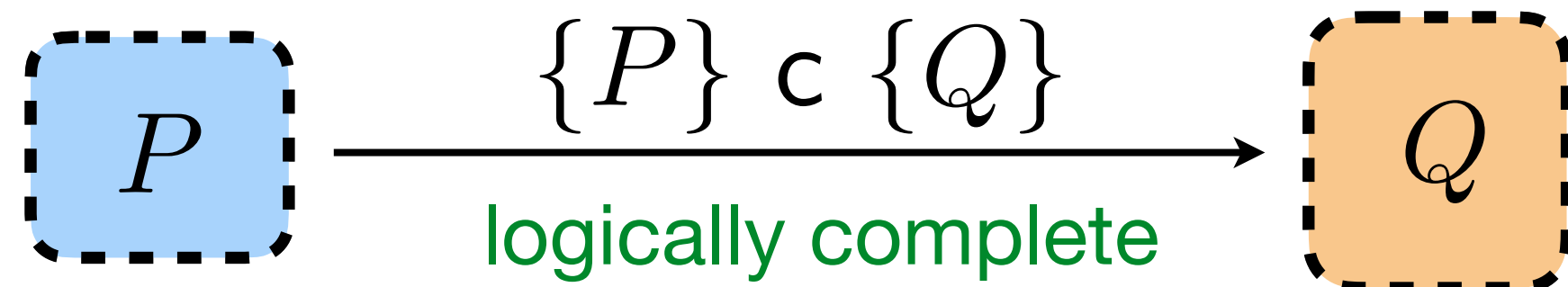
See full version of LICS 2001 paper in JACM



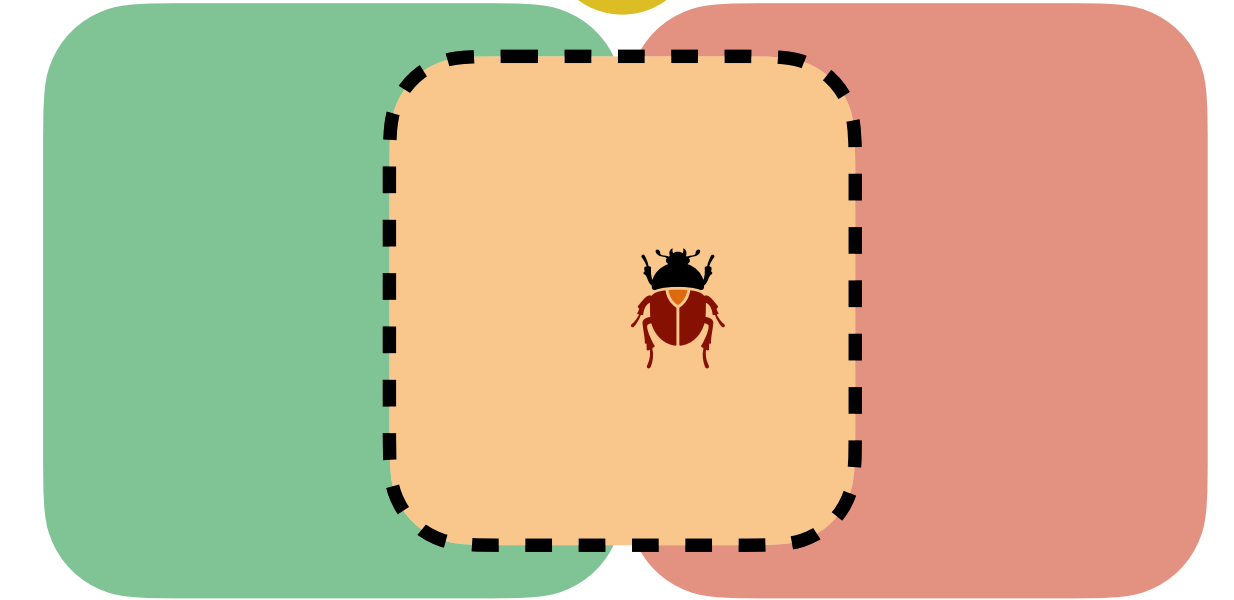
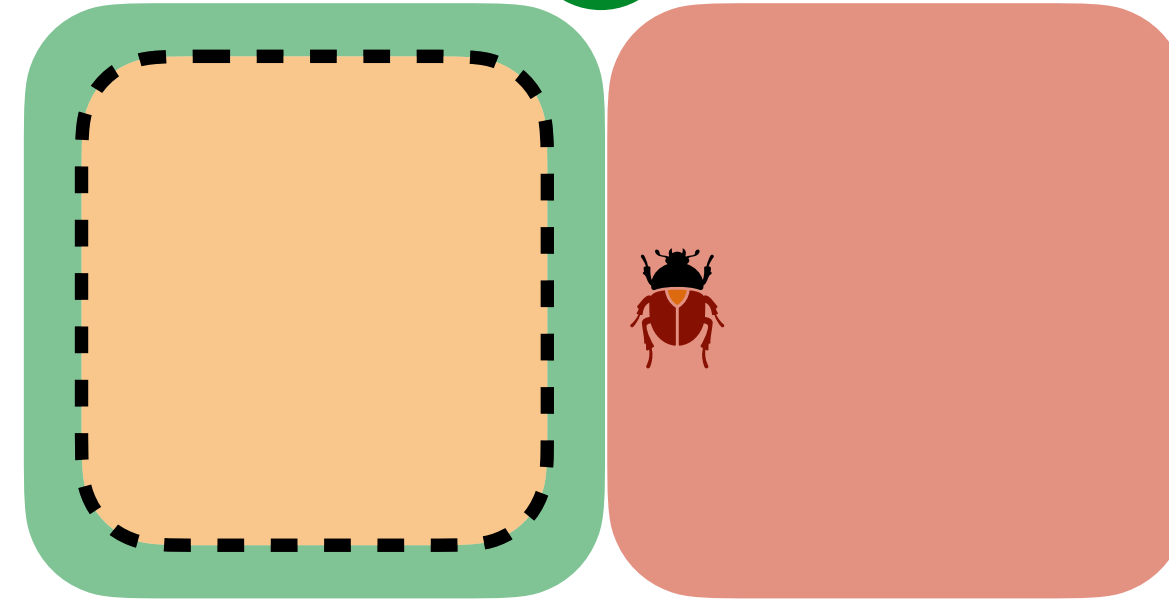
Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Francesco Ranzato:
A Correctness and Incorrectness Program Logic. J. ACM 70(2): 15:1-15:45 (2023)

Over vs Under

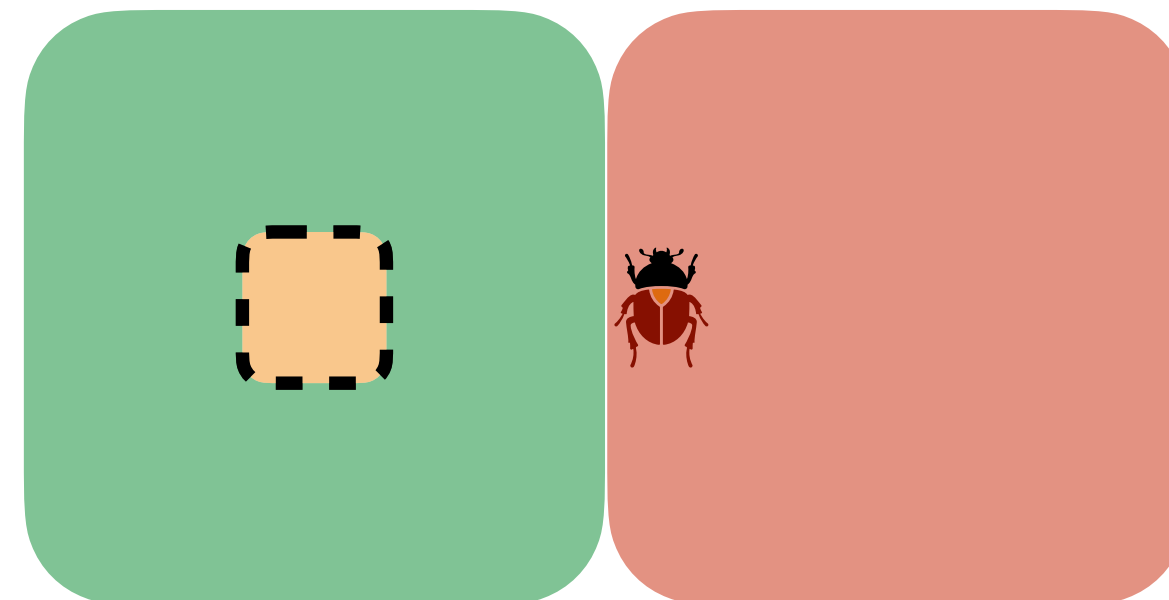
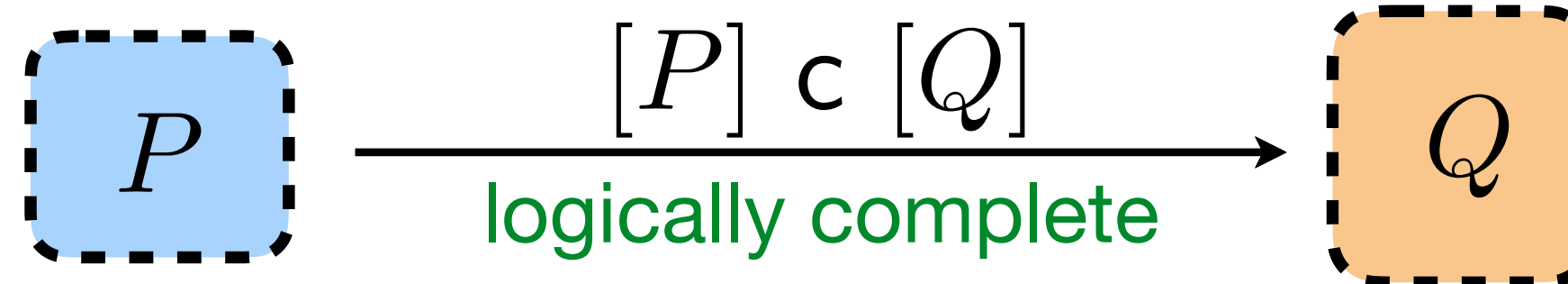
HL



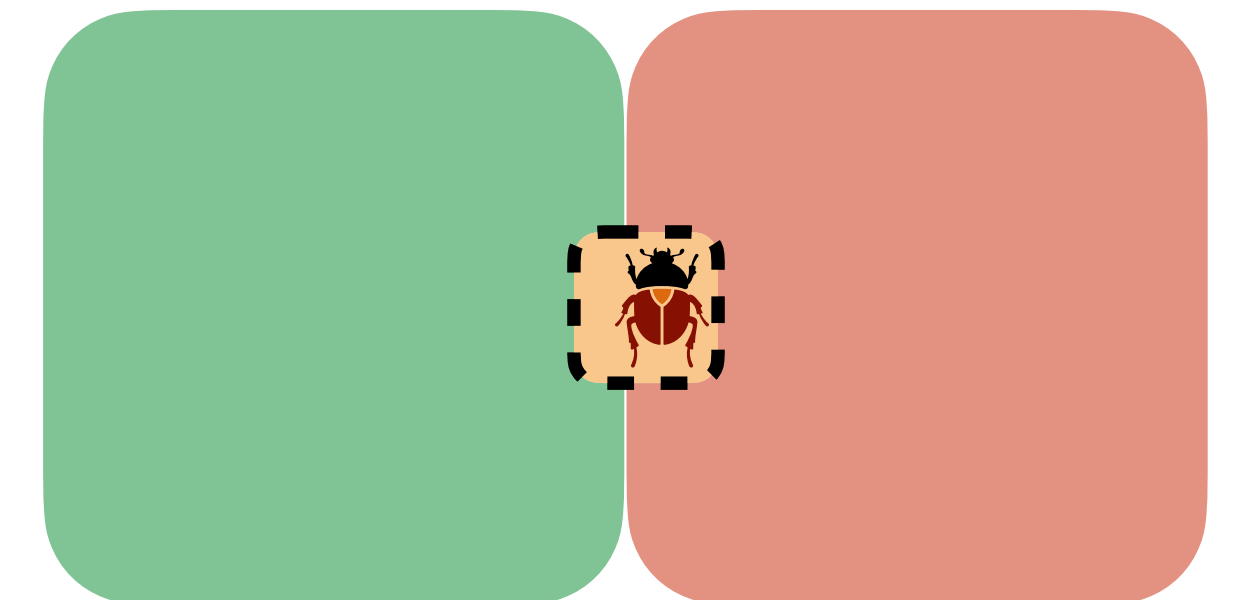
correctness



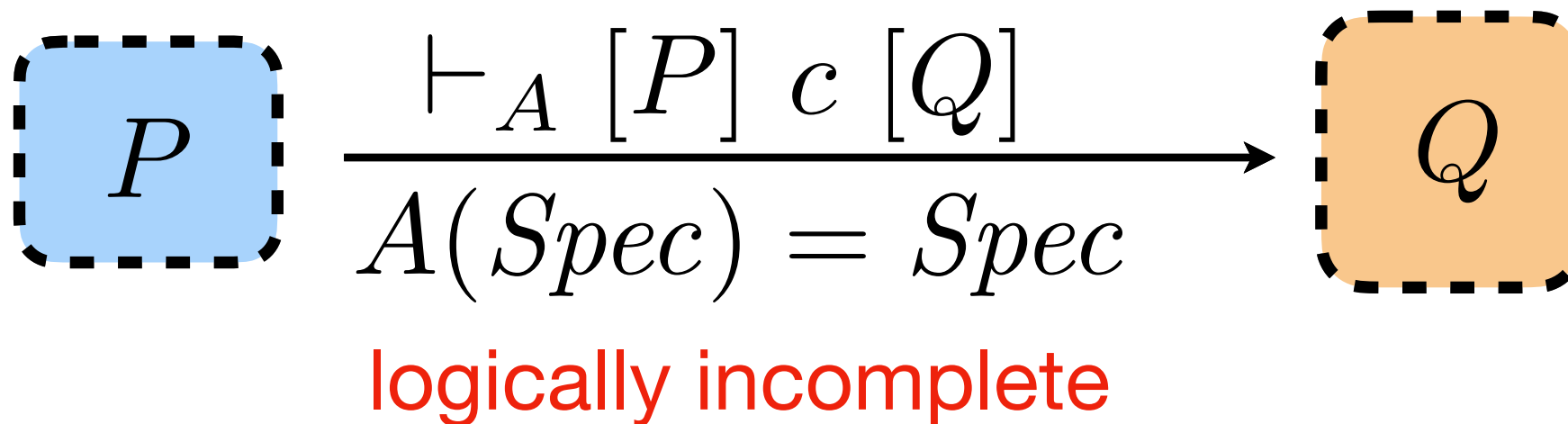
IL



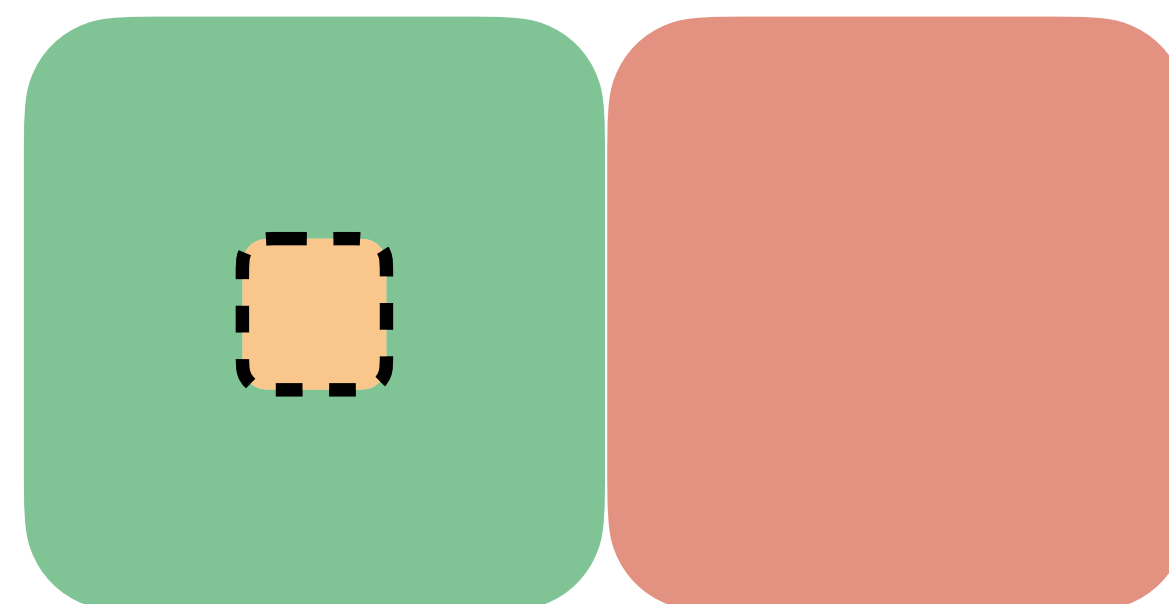
incorrectness



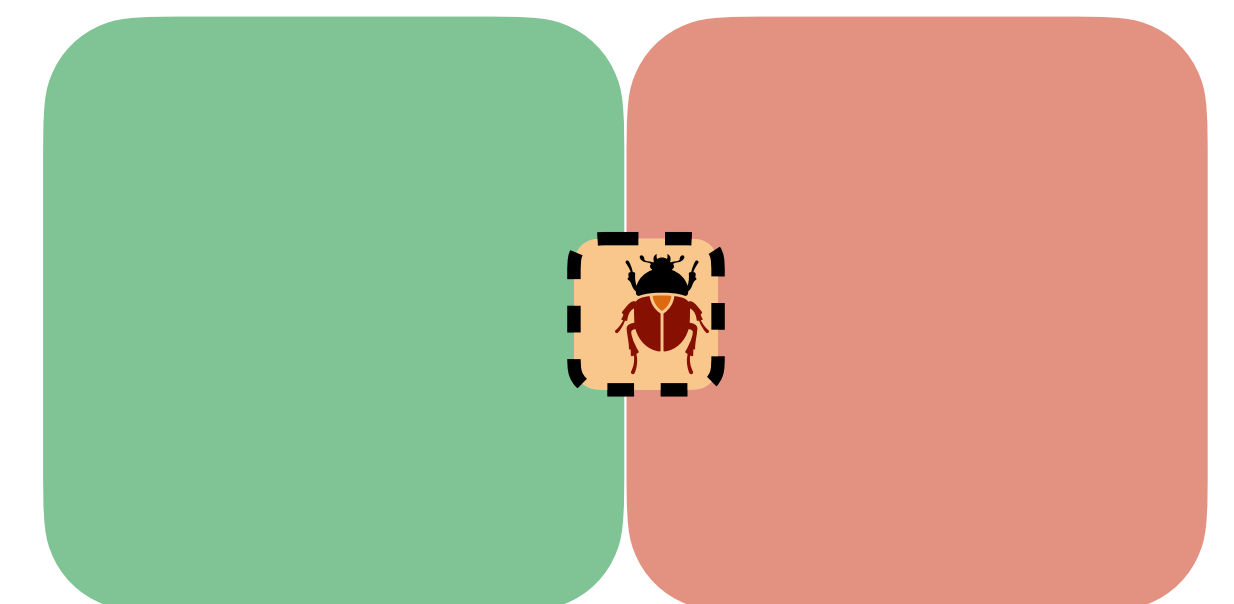
LCL



correctness



incorrectness



What can go wrong?

$$\begin{array}{c}
 \text{local completeness requirement} \quad \text{local completeness requirement} \quad \text{local completeness requirement} \\
 \frac{\mathbb{C}_p^{\text{Sign}^+}(x \leq 0?)}{\vdash_{\text{Sign}^+}[p] \ x \leq 0? \ [-10, -1]} \quad (\text{transfer}) \quad \frac{\mathbb{C}_{\{-10, -1\}}^{\text{Sign}^+}(x := x * 10)}{\vdash_{\text{Sign}^+}[\{-10, -1\}] \ x := x * 10 \ [-100, -10]} \quad (\text{transfer}) \\
 \hline
 \vdash_{\text{Sign}^+}[p] \ x \leq 0?; x := x * 10 \ [-100, -10] \quad \{-100, -10\} \subseteq \text{Sign}^+(p) = \mathbb{Z}_{\neq 0} \quad (\text{seq}) \\
 \hline
 \vdash_{\text{Sign}^+}[p] \ (x \leq 0?; x := x * 10)^* \ [-100, -10, -1, 100] \quad \{-100, 100\} \subseteq \{-100, -10, -1, 100\} \subseteq \text{Sign}^+(\{-100, 100\}) = \mathbb{Z}_{\neq 0} \quad (\text{iterate}) \\
 \hline
 \vdash_{\text{Sign}^+}[p] \ (x \leq 0?; x := x * 10)^* \ [-100, 100] \quad (\text{relax}) \quad \frac{\mathbb{C}_{\{-100, 100\}}^{\text{Sign}^+}(0 < x?)}{\vdash_{\text{Sign}^+}[\{-100, 100\}] \ 0 < x? \ [100]} \quad (\text{transfer}) \\
 \hline
 \vdash_{\text{Sign}^+}[p] \ c \ [100] \quad (\text{seq})
 \end{array}$$

local-completeness proof obligations can fail!

any non-trivial abstract domain A introduces some imprecision!

Exercises

Validity

Which LCL triples are valid for any c and P ?

$$\vdash_{\{\top\}} [P] \ c \ [\text{false}]$$

$$\vdash_{\{\top\}} [P] \ c \ [\text{true}]$$

$$\vdash_{\text{Sign}} [x > 10] \ c \ [\text{false}]$$

$$\vdash_{\{\top\}} [wlp(r, P)] \ c \ [P]$$

Validity

Which LCL triples are valid for any c and P ?

$\vdash_{\{\top\}} [P] c [\text{false}]$



$\vdash_{\{\top\}} [P] c [\text{true}]$



$\vdash_{\text{Sign}} [x > 10] c [\text{false}]$



$\vdash_{\{\top\}} [wlp(r, P)] c [P]$



Reasoning

Find a derivation for the IL triple

$\vdash_{\text{Oct}} [x < 10, y > 20] \text{ if } x \geq y \text{ then } z := x \text{ else } z := y [x < 10, y > 20, z = \max(x, y)]$

$[x < 10, y > 20]$

if $x \geq y$ then

$z := x$

else

$z := y$

$[x < 10, y > 20, z = \max(x, y)]$

Reasoning

Find a derivation for the IL triple

$\vdash_{\text{Oct}} [x < 10, y > 20] \text{ if } x \geq y \text{ then } z := x \text{ else } z := y [x < 10, y > 20, z = \max(x, y)]$

$[x < 10, y > 20]$

if $x \geq y$ then

$[\text{false}]$

$z := x$

$[\text{false}]$

else

$[x < 10, y > 20]$

$z := y$

$[x < 10, z = y > 20] \equiv [x < 10, y > 20, z = \max(x, y)]$

$[x < 10, y > 20, z = \max(x, y)]$

Soundness

Are these “mixed” HL+LCL inference rules valid ?

$$\frac{\vdash_A [P] \text{ } c \text{ } [Q]}{\{P\} \text{ } c \text{ } \{A(Q)\}}$$

$$\frac{\vdash_A [P] \text{ } c \text{ } [Q]}{\{A(P)\} \text{ } c \text{ } \{A(Q)\}}$$

Soundness

Are these “mixed” HL+LCL inference rules valid ?

$$\frac{\vdash_A [P] \text{ } c \text{ } [Q]}{\{P\} \text{ } c \text{ } \{A(Q)\}}$$

$$\frac{\vdash_A [P] \text{ } c \text{ } [Q]}{\{A(P)\} \text{ } c \text{ } \{A(Q)\}}$$

If $\vdash_A [P] \text{ } c \text{ } [Q]$ then $\llbracket c \rrbracket P \subseteq A(Q)$, hence $\{P\} \text{ } c \text{ } \{A(Q)\}$ is valid

If $\vdash_A [P] \text{ } c \text{ } [Q]$ then $\llbracket c \rrbracket A(P) \subseteq \llbracket c \rrbracket^\# A(P) = A(Q)$, hence $\{A(P)\} \text{ } c \text{ } \{A(Q)\}$ is valid

Unsoundness 1

Prove that [conj] is **unsound** for LCL

$$\frac{\vdash_A [P_1] \text{ c } [Q_1] \quad \vdash_A [P_2] \text{ c } [Q_2]}{\vdash_A [P_1 \wedge P_2] \text{ c } [Q_1 \wedge Q_2]} \text{ [conj]}$$

Unsoundness 1

Prove that [conj] is **unsound** for LCL

$$\frac{\vdash_A [P_1] \text{ c } [Q_1] \quad \vdash_A [P_2] \text{ c } [Q_2]}{\vdash_A [P_1 \wedge P_2] \text{ c } [Q_1 \wedge Q_2]} \text{ [conj]}$$

Consider $\vdash_{\{\top\}} [x = 0] \ x := 1 \ [x = 1]$

and $\vdash_{\{\top\}} [x = 1] \ x := 1 \ [x = 1]$

By rule [conj] we could derive $\vdash_{\{\top\}} [\text{false}] \ x := 1 \ [x = 1]$

which is not sound!

Unsoundness 2

Show that the following rule is not sound

$$\frac{}{\vdash_A [P] x := \text{nondet}() [P[v/x]]}$$

arbitrary
value

Unsoundness 2

Show that the following rule is not sound

$$\frac{}{\vdash_A [P] x := \text{nondet}() [P[v/x]]}$$

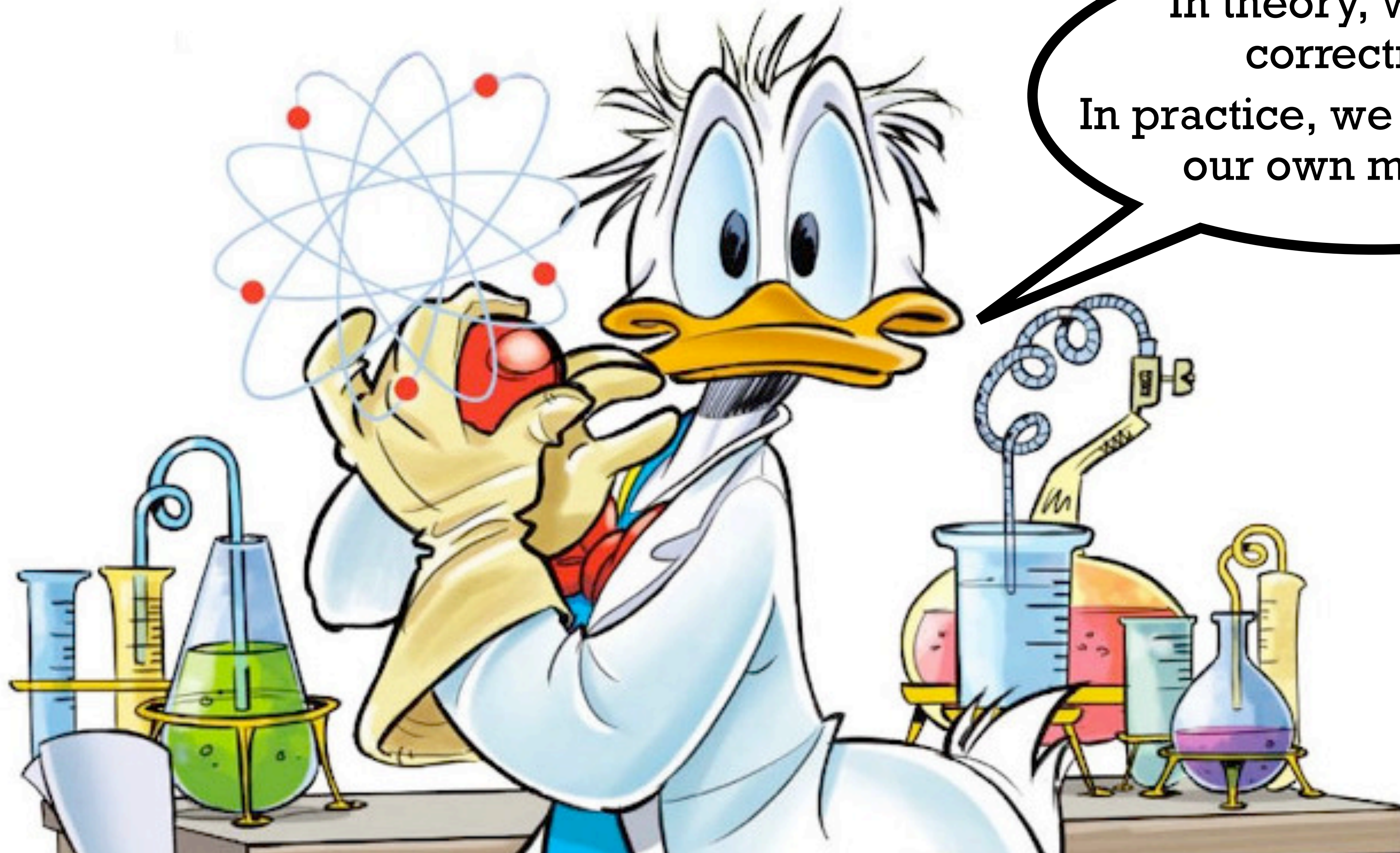
arbitrary
value

Consider $\vdash_{\text{Id}} [x = 0] x := \text{nondet}() [1 = 0] \equiv [\text{false}]$

which is not sound because $\text{Id}(\text{false}) = \perp$ and

$\text{Id}(\llbracket x := \text{nondet}() \rrbracket (x = 0)) = \top$

Take home message



In theory, we prove
correctness.
In practice, we approximate
our own mistakes.